

LC 1992

Citrix

Amazon

Find All Groups of Farmland

© Authored by Arpit Raj, LNMIIT Jaipur, July 22, 2026

Problem Notes & Strategy

Example:

```
1 0 0 → (0,0,0,0)
0 1 1
0 1 1 → (1,1,2,2)
```

- For every farm land, we need to return coordinates of top left and bottom right corner.
- # Since it's given that every farmland is rectangular, if it was arbitrary → BFS, DFS
- # So whenever Aadya begins and stumbles upon 1 → If rectangle → use rectangle property.
- she starts BFS, DFS and track min row, min col, max row, max col.
- # let's say she stumbles upon a land marked 1, then she already got min r = i, min c = j.
- # This for generally any arbitrary shape → now do the whole bfs and dfs to get max r, and max c and push to ans.
- You can also use rectangle traversal property, which is there in notes.

Solution 1: Rectangle Traversal Property

```
class Solution {
public:
    vector<vector<int>> findFarmland(vector<vector<int>>& land) {
        int n = land.size();
        int m = land[0].size();
        vector<vector<int>> ans;

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(land[i][j] == 1) {
                    if((i == 0 || land[i-1][j] == 0) && (j == 0 || land[i]
[j-1] == 0)) {
                        int r = i;
                        int c = j;

                        while(r + 1 < n && land[r+1][j] == 1) {
                            r++;
                        }

                        while(c + 1 < m && land[i][c+1] == 1) {
                            c++;
                        }

                        ans.push_back({i, j, r, c});
                    }
                }
            }
        }
        return ans;
    }
};
```

Solution 2: Breadth-First Search (BFS)

```
class Solution {
public:
    vector<vector<int>> findFarmland(vector<vector<int>>& land) {
        int n = land.size();
        int m = land[0].size();
        vector<pair<int,int>> dir = {{-1,0}, {1,0}, {0,1}, {0,-1}};
        queue<pair<int,int>> q;

        int maxr, maxc, minr, minc;
        vector<vector<int>> ans;

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(land[i][j] == 1) {
                    maxr = INT_MIN;
                    maxc = INT_MIN;
                    land[i][j] = 0;
                    q.push({i, j});
                    minr = i;
                    minc = j;

                    while(!q.empty()) {
                        int r = q.front().first;
                        int c = q.front().second;
                        maxr = max(maxr, r);
                        maxc = max(maxc, c);
                        q.pop();

                        for(auto it : dir) {
                            int nr = r + it.first;
                            int nc = c + it.second;

                            if(nr >= 0 && nc >= 0 && nr < n && nc < m) {
                                if(land[nr][nc] == 1) {
                                    land[nr][nc] = 0;
                                    q.push({nr, nc});
                                }
                            }
                        }
                    }
                    ans.push_back({minr, minc, maxr, maxc});
                }
            }
        }
        return ans;
    }
};
```

Solution 3: Depth-First Search (DFS)

```
class Solution {
public:
    vector<pair<int,int>> dir = {{1,0}, {-1,0}, {0,1}, {0,-1}};
    int n, m;

    void dfs(int r, int c, vector<vector<int>>& land, int &maxr, int
&maxc) {
        land[r][c] = 0;
        maxr = max(maxr, r);
        maxc = max(maxc, c);

        for(auto it : dir) {
            int nr = r + it.first;
            int nc = c + it.second;

            if(nr >= 0 && nr < n && nc >= 0 && nc < m) {
                if(land[nr][nc] == 1) {
                    dfs(nr, nc, land, maxr, maxc);
                }
            }
        }
    }

    vector<vector<int>> findFarmland(vector<vector<int>>& land) {
        n = land.size();
        m = land[0].size();
        vector<vector<int>> ans;

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < m; j++) {
                if(land[i][j] == 1) {
                    int maxr = i;
                    int maxc = j;

                    dfs(i, j, land, maxr, maxc);
                    ans.push_back({i, j, maxr, maxc});
                }
            }
        }
        return ans;
    }
};
```